

[63:32] 或 [31:0] 位。

6.2.1.2 调用 Xilinx IP 实现除法运算部件

我们这里不使用“/”和“%”这类运算符来让综合工具推导实现除法运算部件，主要原因是 Vivado 综合工具将用 LUT 实现一个单周期的除法运算部件，其时序会非常差。接下来介绍的是基于交互界面定制除法器 IP 的方法。

在工程导航栏中点击“IP Catalog”，然后在右侧出现的窗口中搜索 div，如图 6.1 所示。

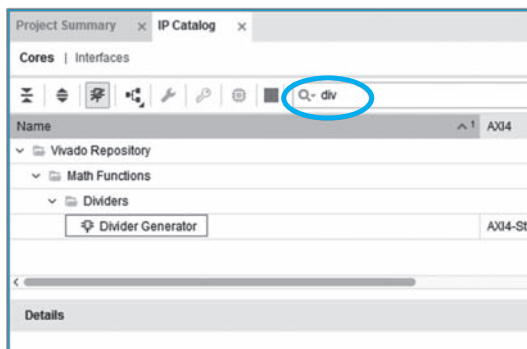


图 6.1 在“IP Catalog”界面中选择“Divider Generator”

在搜索结果中双击“Divider Generator”项，打开除法器 IP 设置界面，依次设置参数，如图 6.2 所示。

对于除法器 IP 设置选项中需要大家关注的地方，我们在图中做了标记和说明。对于除法器的名字，读者可以根据自己的喜好进行调整，并不一定要采用图中的示例名。在“Channel Settings”选项卡中，我们首先要对除法器 IP 的算法进行选择，这里建议采用 Radix2 算法。采用 Radix2 算法实现的除法器的优点是资源消耗少，缺点是完成运算需要的迭代周期数多。通常，应用程序中出现定点除法运算的比例很低，而且编译器一般会尽可能用加、减、移位等指令来实现除法运算，所以除法指令实现的周期数略多一些，但对应用的平均性能不会造成太大的影响。在“Channel Settings”选项卡中还可以选择除法运算是有符号除法还是无符号除法。与前面介绍的乘法器 IP 类似，这里也需要生成有符号和无符号两个除法器 IP，才能分别用于实现 div.w/mod.w 和 div.wu/mod.wu 指令。接下来的 5、6 两处分别定义被除数和除数的位宽为 32 位。在 7 处，一定要将 Remainder Type 选择为 Remainder，以确保余数类型是整型。我们不用勾选 8 处的除 0 检测，因为 LoongArch 指令规范中规定除法指令自身是不需要对除数为 0 进行特

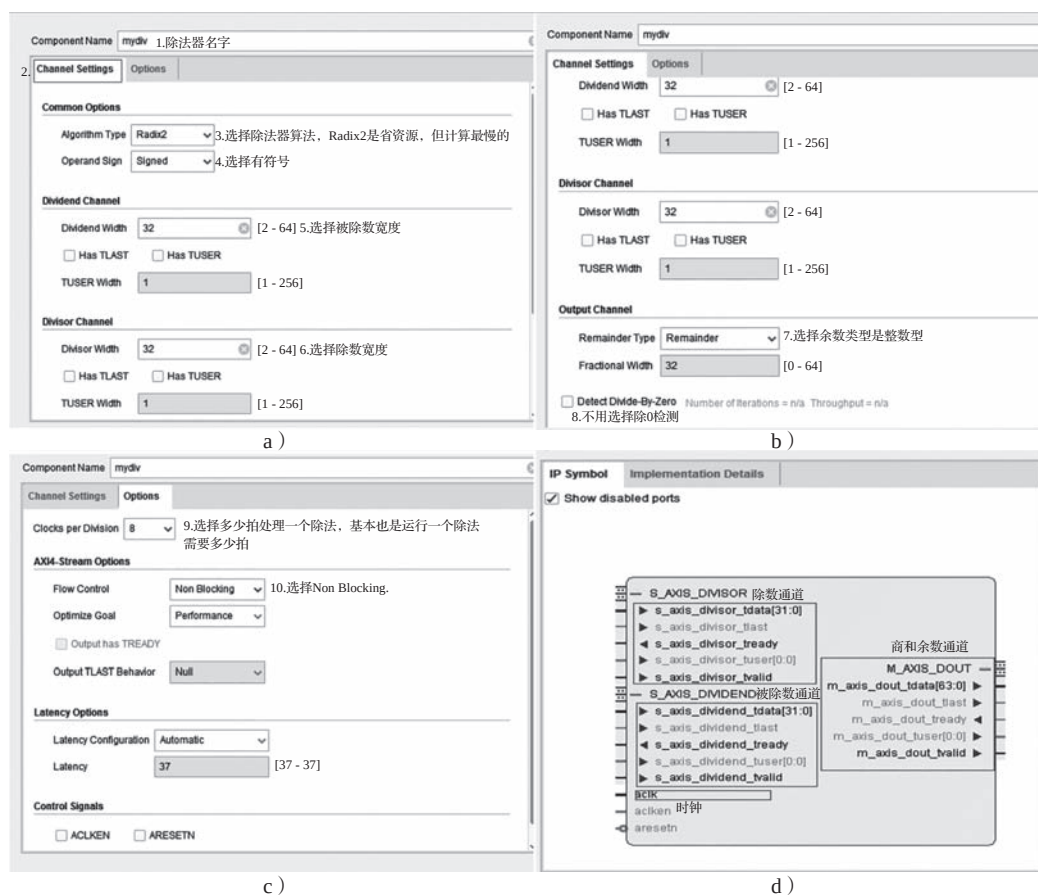


图 6.2 设置除法器 IP 核的参数

殊处理的, 这项工作交给软件去做。在“Options”选项卡中, 9 处选择多少周期处理一个除法, 这个拍数是连续处理多个除法运算之间的间隔周期数, 不是完成一个除法运算的周期数。完成一个除法运算完成的周期数由图 6.2c 中的“Latency Options”参数决定。10 处 AXI 接口上的流控可以按照我们的建议选择 Non Blocking, 这样选择意味着除法器产生计算结果后, 外部一定能够取走这个结果, 不会阻塞结果的输出。在我们的单发射静态流水线中, 这个条件是始终成立的。(请读者自己想一想其中的原因。)

尽管选择 Non Blocking 的流控策略, 但由于目前 Vivado 中提供的除法器 IP 一定是 AXI 接口的, 因此接下来我们还是要对模块顶层接口的相关信号进行一些介绍。观察图 6.2d, 总体上我们会看到被除数、除数通道以及商和余数通道。其中, 被除数通道中的 32 位输入信号 `s_axis_dividend_tdata` 对应计算时输入的被除数数据; 除数通道中的 32 位输入信号 `s_axis_divisor_tdata` 对应计算时输入的除数数据; 计算得到的商和余数

结果统一从商和余数通道中的 64 位信号 `m_axis_dout_tdata` 中输出，其中第 [63:32] 位存放的是商，第 [31:0] 位存放的是余数。

对于两个输入通道和一个输出通道，除了有上述数据信号外，每个通道都有一对 `tvalid`、`tready` 信号。这是一对“握手”控制信号，其工作原理类似于我们在 CPU 流水线之间使用的 `valid`、`allowin` 信号。`tvalid` 是请求信号，`tready` 是应答信号。在时钟上升沿到来时，如果采样得到 `tvalid` 和 `tready` 都等于 1，则请求发起方和接收方之间完成一次成功的握手。如果假想接收方有一组触发器缓存，那么所谓的成功握手是指发送方的数据写入接收方的缓存中，也就是在握手成功的这个上升沿之后，触发器缓存会变为发送方的数据。

假设在执行流水阶段调用所生成的除法器 IP。在除法指令处于执行流水级且没有对除法器成功输入数据的时候，同时将 `s_axis_dividend_tvalid` 和 `s_axis_divisor_tvalid` 置为 1。当发现 `s_axis_dividend_tready` 和 `s_axis_divisor_tready` 反馈为 1 后（此时在一个时钟上升沿同时看到 `tvalid` 和 `tready` 为 1，表示握手成功），需要将 `s_axis_dividend_tvalid` 和 `s_axis_divisor_tvalid` 清 0，也就是确保握手成功的那个时钟上升沿之后的 `s_axis_dividend_tvalid` 一定同时置为 1，那么这里生成的除法器 IP 反馈的 `s_axis_dividend_tready` 和 `s_axis_divisor_tready` 一定也同时置为 1。这里再次强调，被除数和除数输入的 `tready` 置起后（也就是握手成功后），`tvalid` 一定要撤销，否则除法器 IP 会认为又有一个新的除法运算。

完成输入数据的握手之后，除法指令就需要在执行流水级等待除法器 IP 最终输出结果。当 `m_axis_dout_tvalid` 置为 1 时，表示除法计算完成。此时除法指令就可以从 `m_axis_dout_tdata` 上取出计算结果，进入流水线的后续阶段。由于前面我们将流控策略设置为 Non Blocking，因此输出通道上不需要外部反馈 `tready` 信号，换言之，不管产生多少结果、什么时候产生，外部都要能够及时处理。

采用本小节介绍的实现方案，尽管除法是在 CPU 的执行流水阶段开始执行并产生运算结果，但由于其运算会花费多个时钟周期，所以流水线的控制上要做一些针对性的调整。具体来说，就是当执行流水阶段上是一条除法指令时，仅当除法运算部件返回结果完成的信号（Xilinx 除法器 IP 的 `m_axis_dout_tvalid` 信号为 1）之后这一级流水的 `ready_go` 才能置为有效。

如果读者不想在 CPU 设计中将更多的时间花费在乘除法运算电路的设计实现上，那么按照这一节介绍的方法去实现就可以了，然后直接跳过接下来的 6.2.2 节和 6.2.3 节。